

Architecture Decision Records

Key Technical Decisions — 2025-2026 Collection

Engineering Architecture Review Board

Maintainer: Dr. Tran Minh Khoa, Principal Engineer

ADR-001: Adoption of Qdrant as Primary Vector Database

Status: Accepted | Date: October 15, 2025 | Deciders: ARB (CTO, VP Eng, Principal Engineers, CISO)

Context and Problem

TechViet is building multiple AI-powered products requiring high-performance vector similarity search: ChatAssist (internal RAG chatbot, ~10M vectors), ShopSmart Reco (product embeddings, ~5M vectors), and LogiFlow AI (location/route embeddings, ~2M vectors). We need a vector database that can handle production workloads with low latency, supports metadata filtering for access control, and can be self-hosted in our AWS ap-southeast-1 infrastructure for data residency.

Evaluation Results

We conducted a 4-week proof-of-concept evaluation with 1M vectors (1024 dimensions, matching our multilingual-e5-large embedding model). Each solution was tested on the same hardware (r6g.xlarge instances) with identical query patterns. The evaluation criteria and weights were agreed upon by the ARB before testing began.

Criteria (Weight)	Qdrant	Pinecone	Weaviate	Milvus	pgvector
Query Latency P95 (25%)	4.2ms — 9/10	3.8ms — 9/10	8.1ms — 7/10	5.5ms — 8/10	22ms — 5/10
Scalability to 10M+ (20%)	Sharding + replication — 8/10	Fully managed, unlimited — 10/10	Auto-sharding — 8/10	Distributed native — 9/10	Limited by PG — 4/10
Cost at Target Scale (20%)	~\$180/mo self-hosted — 9/10	~\$850/mo managed — 4/10	~\$250/mo self-hosted — 7/10	~\$220/mo self-hosted — 7/10	~\$50/mo (in existing PG) — 10/10
Metadata Filtering (15%)	Native payload filters, fast — 9/10	Basic filters, limited ops — 7/10	GraphQL-style filters — 8/10	Boolean expressions — 8/10	SQL WHERE clause — 6/10
Self-hosted Capable (10%)	Docker/K8s, easy — 10/10	SaaS only — 0/10	Docker/K8s — 10/10	K8s, complex setup — 10/10	Same as PG — 10/10
Docs & Community (10%)	Good docs, growing — 8/10	Excellent — 9/10	Decent — 7/10	Large but fragmented — 7/10	PG ecosystem — 8/10
WEIGHTED TOTAL	8.75 / 10	6.55 / 10	7.60 / 10	7.90 / 10	6.30 / 10

Decision

Adopt Qdrant as the primary vector database, deployed as a 3-node cluster on EKS with NVMe-backed EBS volumes. Pinecone is approved as a secondary option for rapid prototyping and PoC work where self-hosting overhead is not justified. pgvector is permitted for projects with small vector collections (<100K vectors) that don't warrant a separate vector database.

Consequences:

- DevOps team must create Helm charts and runbooks for Qdrant cluster operations
- ML team to develop shared Python SDK for Qdrant interactions (techviet-vectordb library)
- Monitoring: Qdrant metrics exported to Datadog; alerting on cluster health and query latency

- Backup: Qdrant snapshots to S3 every 6 hours; tested restore procedure monthly

ADR-002: Strangler Fig Migration from Django Monolith to Microservices

Status: Accepted | Date: November 5, 2025 | Deciders: ARB + VietBank Project Leads

Context and Problem

The Phoenix Platform (VietBank's core banking modernization project) has grown to 280,000 lines of code in a single Django 4.2 monolith over 18 months of development. The monolith has become a significant impediment to delivery speed and reliability:

- Deployment frequency dropped from daily to bi-weekly (entire app must be deployed together)
- A single failing test in any module blocks all deployments for all teams
- Database schema changes require coordination across 5 teams, causing frequent merge conflicts
- Build time: 25 minutes (was 4 minutes 12 months ago). Test suite: 47 minutes.
- Scaling is all-or-nothing: cannot scale the transaction processing module independently

Decision

Adopt the Strangler Fig pattern to incrementally extract high-value services from the monolith. The monolith remains operational throughout migration — no big-bang rewrite. Services are extracted based on a prioritization score combining business criticality, change frequency, team coupling, and scaling requirements.

Phase	Service to Extract	Size	Target Stack	Timeline	Team	Priority Score
1	Authentication & Authorization	12K LOC	Go + gRPC + Redis	Q1 2026	Platform Team (4 eng)	9.2 — highest change frequency; security-critical
2	Notification Service	8K LOC	Python + FastAPI + Kafka	Q1 2026	Integrations (3 eng)	8.5 — low coupling; easy extraction; blocking email reliability
3	Transaction Processing	45K LOC	Go + Kafka + PostgreSQL	Q2 2026	Core Banking (6 eng)	8.8 — scaling bottleneck; highest business value
4	Reporting & Analytics	35K LOC	Python + FastAPI + ClickHouse	Q2-Q3 2026	Data Team (4 eng)	7.5 — performance issues; separate data store beneficial
5	Customer Management	28K LOC	Go + gRPC + PostgreSQL	Q3 2026	Core Banking (4 eng)	7.0 — moderate coupling; benefits from independent scaling
6	Remaining Monolith	152K LOC	Django 4.2 (maintained)	Ongoing	All teams (shared)	N/A — gradually shrinks as services extracted

Consequences:

- Need service mesh (Istio) for inter-service communication, observability, and mTLS
- API gateway (Kong) to route traffic between monolith and new services during migration
- Distributed tracing (Datadog APM) critical for debugging cross-service requests
- Each extraction requires: data migration plan, backward-compatible API, 2-week parallel run

ADR-003: Standardizing on GitHub Actions for All CI/CD

Status: Accepted | Date: December 1, 2025 | Deciders: ARB + DevOps Team

Context and Problem

TechViet currently operates 3 separate CI/CD systems: Jenkins (8 projects, self-hosted on EC2), GitLab CI (2 projects, SaaS), and GitHub Actions (14 projects). This fragmentation creates significant operational overhead: 3 systems to maintain and secure, inconsistent pipeline definitions, different secret management approaches, and difficulty enforcing company-wide security scanning standards. The Jenkins server requires approximately 8 hours/month of DevOps maintenance and has suffered 3 outages in the last 6 months.

Evaluation Criteria	Jenkins	GitLab CI	GitHub Actions
Maintenance Effort	HIGH: Self-hosted on EC2; 8 hrs/mo maintenance; frequent plugin updates; Java upgrades required	LOW: SaaS managed; but separate from our primary Git platform	LOW: SaaS managed; native to our Git platform (GitHub Enterprise)
Integration Quality	MEDIUM: Plugin-based GitHub integration; can be unreliable; requires webhooks	LOW: Requires mirroring repos from GitHub to GitLab; adds complexity and latency	HIGH: Native integration; status checks, PR annotations, Dependabot, CodeQL built-in
Ecosystem / Marketplace	LARGE but aging: Many plugins are unmaintained; security risk from unvetted plugins	GROWING: Good built-in features but smaller marketplace than GitHub Actions	LARGEST: 15,000+ actions; active community; first-party actions for major tools
Security Scanning	MANUAL: Must configure Snyk, SonarQube plugins separately; inconsistent across projects	BUILT-IN: Basic SAST/DAST; limited compared to GitHub's offering	EXCELLENT: CodeQL, Dependabot, secret scanning native; Snyk integration seamless
Self-hosted Runners	N/A (all self-hosted)	AVAILABLE: Works but management overhead	AVAILABLE: Using for builds; well-documented; auto-scaling with Actions Runner Controller
Cost (Annual)	~\$12,000 (EC2 + maintenance time)	~\$5,400 (SaaS plan for 2 projects)	Included in GitHub Enterprise license (\$0 incremental)
Current Projects	8 (all legacy)	2 (will migrate to GitHub)	14 (all new projects)

Decision

Standardize on GitHub Actions for all CI/CD pipelines. Create a shared library of reusable workflow templates in the techviet/ci-templates repository. Migration plan: GitLab CI projects migrated by end of Q1 2026 (2 projects, low risk). Jenkins projects migrated by end of Q2 2026 (8 projects, staged migration). Jenkins EC2 instances decommissioned by July 2026.